

# Módulo 17: Empaquetado, distribución y CI/CD

## Contenidos clave

### **Construcción de ruedas (wheels) y publicación (PyPI/artefactos internos):**

- Wheel: (.whl) una wheel es un formato de distribución de paquetes Python. El archivo puede instalarse y sirve principalmente para distribuir software Python de manera reproducible y sencilla.
- PyPI: (Python Package Index) es el repositorio público donde se distribuyen paquetes Python. Cuando se hace “pip install fastapi” normalmente pip obtiene el paquete desde PyPI. También existen registries privados para empresas.
- Artefactos reproducibles: un artefacto es un resultado generado por el proceso de construcción. La idea de reproducible significa que, utilizando el mismo código y las mismas dependencias/versiones, se pueda obtener esencialmente el mismo resultado.

**Docker multistage y buenas prácticas:** Docker Multistage Build. Consiste en utilizar diferentes etapas para construir la aplicación y después ejecutar únicamente lo necesario. La imagen final puede ser más limpia porque no necesita herramientas de compilación que solo fueron utilizadas durante el build.

Buenas prácticas de Docker:

- Usar imágenes pequeñas.
- No ejecutar como root.
- No guardar secretos
- Usar .dockerignore

### **Pipelines en GitHub Actions/Azure DevOps:**

- CI/CD: CI = Continuous Integration, CD = Continuous Delivery/Deployment.  
Developer -> git push -> Pipeline -> Lint -> Type-check -> Tests -> Build -> Deploy.
- CI: Cada vez que se hace un push o se abre un Pull Request, automáticamente se ejecutan comprobaciones: git push -> GitHub Actions -> Ruff -> Mypy -> Pytest, si falla el pipeline falla, pero si todo funciona el pipeline pasa.
- CD: Después de comprobar que el código funciona, podemos automatizar su entrega: CI -> Tests -> Build -> Docker image -> Push al registry -> Azure -> Deploy.
- Docker Registry: Un container registry es como un repositorio para imágenes Docker. Una opción común es Azure Container Registry (ACR). El pipeline puede hacer: `docker build -t orders-api:1.0.0 .`, después: `docker push registry/orders-api:1.0.0` y Azure puede utilizar esa imagen para ejecutar la aplicación.

### **Despliegue en Azure (App Service, Container Apps, Functions):**

- Azure App Service: Es un servicio administrado para ejecutar aplicaciones web y APIs. Puede desplegar una aplicación sin administrar directamente un servidor. Es apropiado para aplicaciones web/API que necesitan un entorno administrado.
- Azure Container Apps: está orientado específicamente a ejecutar aplicaciones basadas en contenedores. Tiene características útiles para aplicaciones contenerizadas, como escalado y revisiones.

- Azure Functions: En lugar de mantener una aplicación web ejecutándose continuamente, se puede ejecutar en funciones en respuesta a eventos: HTTP request -> Azure Function -> procesar\_orden(). Este modelo es útil para tareas/eventos específicos.

## Objetivos

### **- Empaquetar, contenerizar y automatizar pipelines.**

Se configuró Poetry en pyproject.toml para que el proyecto funcione en modo paquete:

```
[tool.poetry]
package-mode = true
packages = [
    { include = "labs" },
]
```

Y el archivo poetry.lock mantiene las versiones exactas de las dependencias.

Se creó un Dockerfile multistage con:

- Etapa builder
- Instalación de Poetry
- Instalación de dependencias desde poetry.lock
- Etapa runtime independiente
- Copia únicamente de lo necesario para ejecutar la aplicación
- Usuario no root para mejorar la seguridad
- Exposición del puerto 8000
- Inicio de FastAPI con Uvicorn

Se configuró .github/workflow/ci.yml para ejecutar automáticamente:

- Validación de código
- Formato
- Imports
- Tipado estático
- Pruebas
- Cobertura
- Construcción de wheel
- Construcción y publicación de Docker.

Además el job Docker depende del job de calidad, esto garantiza que la imagen solo se publique después de superar todas las validaciones.

### **- Publicar artefactos reproducibles.**

Se aseguró mediante:

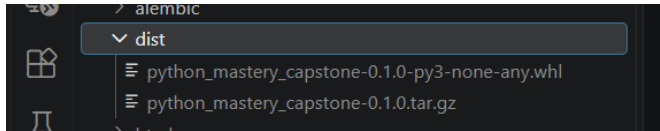
- pyproject.toml para declarar el proyecto y sus dependencias
- poetry.lock para fijar las versiones exactas
- Imagen base explícita
- wheel generada a partir de la misma configuración
- workflow automatizado en GitHub Actions

- Imagen etiquetada y publicada en GHCR

## Laboratorio

### - *Generar wheel*

La wheel se generó con poetry build, se obtuvieron:



La wheel contiene el código del proyecto y los metadatos de distribución.

### - *Dockerfile multistage para FastAPI*

El Dockerfile se dividió en dos etapas:

- Etapa builder: Instala poetry, copia pyproject.toml, poetry.lock y [README.md](#), verifica el proyecto, instala las dependencias principales y copia el código.
- Etapa runtime: En esta etapa se copian:

```
COPY --from=builder /usr/local/lib/python3.14/site-packages \
    /usr/local/lib/python3.14/site-packages
```

```
COPY --from=builder /usr/local/bin \
    /usr/local/bin
```

```
COPY --from=builder /build/labs ./labs
```

Se crea un usuario no root y se inicia FastAPI.

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python> docker run --rm orders-api:local id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python> 
```

La imagen se construyó mediante: `docker build -t orders-api:local`.

```
Problems Output Debug Console Terminal Ports
#14 DONE 0.2s
#15 [runtime 5/6] COPY --from=builder /build/labs ./labs
#15 DONE 0.1s
#16 [runtime 6/6] RUN useradd --create-home --shell /usr/sbin/nologin appuser && chown -R appuser:appuser /app
#16 DONE 1.5s
#17 exporting to image
#17 exporting layers
#17 exporting layers 19.8s done
#17 exporting manifest sha256:d664ee0d4155c1709dc9c8da276cbded16bab85281c0759fa37e53f3d46939fc 0.0s done
#17 exporting config sha256:b04bbcbdc385a3780e1e574533d731e5950e2e68bb722f043eccc5dc8c8ca2c 0.0s done
#17 exporting attestation manifest sha256:5cc1cd83d12e78d319e6a7e898cd70013033a42ac6744b196b4bac23eeb73d65
#17 exporting attestation manifest sha256:5cc1cd83d12e78d319e6a7e898cd70013033a42ac6744b196b4bac23eeb73d65 0.1s done
#17 exporting manifest list sha256:b049958d5ce88e5890c7f675ed5f9a540fddc31b895ac0a409342a1e8c590a99 0.1s done
#17 naming to docker.io/library/orders-api:local
#17 naming to docker.io/library/orders-api:local done
#17 unpacking to docker.io/library/orders-api:local
#17 unpacking to docker.io/library/orders-api:local 6.9s done
#17 DONE 27.0s
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/668u8213qpnabn1r3sd3uzyee
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

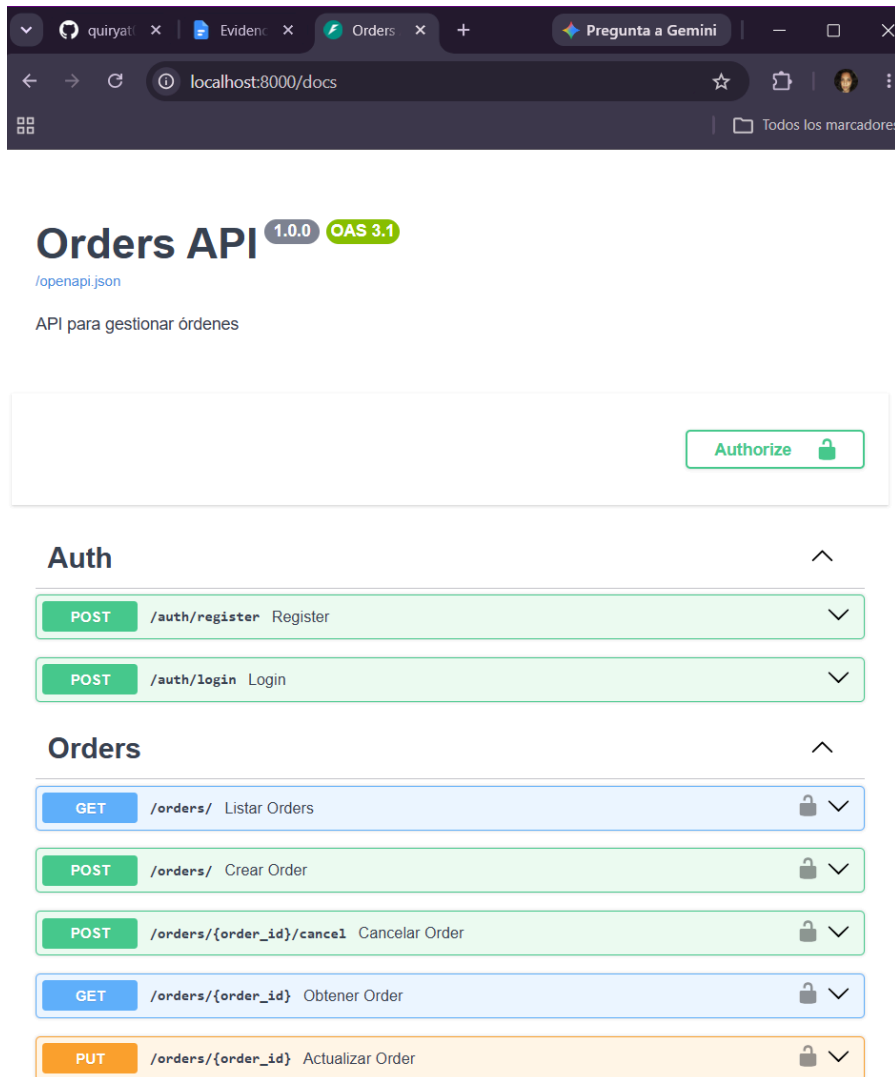
Y se ejecutó con: `docker run --rm -p 8000:8000 orders-api:local`

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> docker run --rm -p 8000:8000 orders-api:local
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Prueba a la API:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python> $response = Invoke-WebRequest -Uri http://localhost:8000/ -UseBasicParsing
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python>
```

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> docker run --rm -p 8000:8000 orders-api:local
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
O: 172.17.0.1:51782 - "GET / HTTP/1.1" 200 OK
INF
```



### - Workflow CI (lint, type-check, tests, build) y push a registry

El workflow quedó definido en ci.yml, contiene:

- Lint con Ruff
- Formato con Black
- Imports con isort
- Type-check con Mypy
- Test y cobertura
- Build de la wheel
- Publicación del artefacto

| Artifacts                                                                                      |         |                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Produced during runtime                                                                        |         |                                                                                                                                                                                                                                                                           |
| Name                                                                                           | Size    | Digest                                                                                                                                                                                                                                                                    |
|  python-wheel | 7.66 MB | sha256:e965d08fe...    |

- Build y push de docker



**All checks have passed**  
2 skipped, 2 successful checks



2 skipped checks 



  CI/CD / docker (pull\_request) Skipp... 

  CI/CD / docker (push) Skipped 2 min... 

2 successful checks 

  CI/CD / quality (pull\_requ... **Required** 

  CI/CD / quality (push) Su... **Required** 



**No conflicts with base branch**  
Merging can be performed automatically.

**Merge pull request** 

You can also merge this with the command line.

[View command line instructions.](#)

← CI/CD

✓ Merge pull request #16 from quiryat09S/feature/day-17-packaging-cicd #54

Re-run all jobs ⋮

Summary

All jobs

✓ quality

✓ docker

Run details

🕒 Usage

📄 Workflow file

Triggered via push 3 minutes ago

Status

👤 quiryat09S pushed

🔗 bf89fd4

main

Success

Total duration

Artifacts

3m 39s

2

ci.yml

on: push

✓ quality 55s

→

✓ docker 2m 38s

🗨

−

+

📦 python-mastery-capstone

main

Public

Latest

Installation

OS / Arch 2

Learn more about packages

📄 Install from the command line

\$ docker pull ghcr.io/quiryat09s/python-mastery-capstone:main

Details

👤 quiryat09S

📦 python-mastery-capstone

☆ 0 stars

Last published

Issues